A D PATEL INSTITUTE OF TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY

SMART JOB SCHEDULING SYSTEM

A Design and Analysis of Algorithms (DAA) Project Report

A greedy job sequencing system that maximises profit while respecting deadlines, with priority-based tie breaking

A report submitted

In partial fulfilment of the requirements for the Degree of

Bachelor of Technology

In

Information Technology

Semester – V

Subject:

Design Analysis Of Algorithm (202045601)

By

MAYURI DUDHAT(12402080601078)

Faculty Guide

Minakshi Patil

Assistant Professor

Academic Year 2026

Department of Information Technology

A D Patel Institute of Technology

New Vallabh Vidyanagar, Vitthal Udyognagar – 388121

Anand (Gujarat), India

CERTIFICATE

This is to certify that the project work embodied in this report entitled “Smart Job Scheduling System: Design and Analysis of Algorithms” was carried out by Mayuri Dudhat (Enrollment No.: 12402080601078) at A D Patel Institute of Technology for partial fulfilment of the requirements of the Bachelor of Technology degree awarded by the Charutar Vidya Mandal (CVM) University. This project work has been carried out under the guidance and supervision of the faculty guide and is to the satisfaction of the department.

Date: _____
Place: New Vallabh Vidyanagar

Lab Teacher	Head of Department
Minakshi Patil	**Dr. Priyang Bhatt**
Assistant Professor	Associate Professor & Head

DECLARATION

I, Mayuri Dudhat (Enrollment No.: 12402080601078), declare that this report on the Smart Job Scheduling System is my own work, prepared for the subject Design and Analysis of Algorithms (202045601) in Semester V of the Bachelor of Technology programme in Information Technology.

The problem formulation, the C++ implementation and the analysis presented here were developed by me while studying the greedy method and job sequencing with deadlines. Wherever I have used ideas, definitions or standard results from textbooks or other sources, they are acknowledged in the References section.

I further declare that the sample outputs shown in this report were produced by running the submitted program, and that no result has been altered or invented.

Date: _____
Place: New Vallabh Vidyanagar

Mayuri Dudhat
(12402080601078)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my subject teacher and department for providing the opportunity to develop and study this project under Design and Analysis of Algorithms. The project gave me a practical way to connect the theory of greedy algorithms with a familiar real-world problem: deciding which tasks to accept when time is limited and every task has a deadline and a reward.

I am thankful to everyone who supported my learning of C++ programming, structures and arrays, sorting techniques and complexity analysis. Their guidance helped me approach the problem systematically, divide the program into small functions, and test how the greedy choice behaves on different sets of jobs.

I also acknowledge the C++ standard library, particularly the iostream, iomanip and string headers, which made it possible to express the program clearly without any external dependencies.

Finally, I thank my classmates and family for their encouragement during the preparation of the project, the presentation and this report.

Mayuri Dudhat
(12402080601078)

TABLE OF CONTENTS

CERTIFICATE **2**
DECLARATION **3**
ACKNOWLEDGEMENT **4**
TABLE OF CONTENTS **5**
ABSTRACT **6**

CHAPTER 1: INTRODUCTION 7
CHAPTER 2: PROBLEM STATEMENT AND OBJECTIVES 8
CHAPTER 3: THEORETICAL BACKGROUND AND THE GREEDY METHOD 9
CHAPTER 4: DATA DESIGN AND JOB MODEL 10
CHAPTER 5: ALGORITHM I: GREEDY JOB SEQUENCING WITH DEADLINES 11
CHAPTER 6: ALGORITHM II: SORTING AND SLOT ALLOCATION 12
CHAPTER 7: SYSTEM DESIGN AND C++ IMPLEMENTATION 13
CHAPTER 8: TESTING AND COMPARATIVE RESULTS 15
CHAPTER 9: SAMPLE EXECUTION AND EVIDENCE 17
CHAPTER 10: COMPLEXITY ANALYSIS AND CORRECTNESS 19
CHAPTER 11: LIMITATIONS, FUTURE SCOPE AND CONCLUSION 21
REFERENCES AND APPENDIX 22

ABSTRACT

The Smart Job Scheduling System is a C++ program that selects and orders jobs so that the total profit is as large as possible while every accepted job finishes on or before its deadline. Each job has an identifier, a name, a deadline, a profit and a priority from 1 to 5. Every job needs exactly one unit of time, and only one job can run in a time slot.

The program uses a greedy algorithm. Jobs are first sorted by profit from high to low, with higher priority breaking ties between equal-profit jobs. Each job is then placed in the latest free slot at or before its deadline; if no such slot exists, the job is skipped. This choice never blocks a more profitable job that can still be accommodated, so the method gives an optimal schedule for the unit-time version of the problem.

The program is menu driven. It can display the job table, generate the schedule, and report the total profit, the number of scheduled and unscheduled jobs, the slot utilisation and the list of jobs that could not be scheduled. A worked example with five jobs produces a schedule of J3, J1 and J5 in slots 1 to 3, a total profit of 142 and a utilisation of 100%.

The report explains the job model, the greedy argument, the pseudocode and C++ implementation, test cases run on the real program, and a complexity analysis. The implementation sorts with bubble sort in $O(n^2)$ time and searches for slots in $O(n \cdot d)$ time, where d is the largest deadline. Replacing the sort with merge sort and the slot search with Union-Find would bring the running time close to $O(n \log n)$.

CHAPTER 1: INTRODUCTION

Scheduling decides which work gets done when time is limited, and this project turns that decision into an algorithm. Operating systems, print queues, factory floors, delivery services and project teams all face the same situation: there are more tasks than time slots, each task has a deadline, and each task is worth a different amount.

A manual comparison of every possible selection quickly becomes impractical. With n jobs there are 2^n possible subsets, and each subset can be ordered in many ways. The Smart Job Scheduling System avoids this explosion by using a greedy strategy: always consider the most profitable job first and give it the latest slot that still meets its deadline.

The project focuses on the job sequencing problem with deadlines, a classic topic in Design and Analysis of Algorithms. It shows why the greedy method works for this problem, how the choice of data structure affects running time, and how a simple tie-breaking rule such as priority makes the output predictable.

The implementation is written in C++ and is menu driven. A user enters up to 20 jobs, views them in a formatted table and generates a schedule. The report then shows the time slots, the total profit, the utilisation of the slots and the jobs that had to be skipped.

The program is an educational model. It handles unit-time jobs only and keeps all data in memory, so it should be read as a demonstration of the algorithm rather than a production scheduler.

CHAPTER 2: PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Given n jobs, each with an ID, a name, a deadline, a profit and a priority, select and order a subset of jobs to maximise the total profit. Every job takes one unit of time, only one job can run in a time slot, and a job earns its profit only if it finishes on or before its deadline. When two jobs have the same profit, the one with the higher priority is considered first.

2.2 Objectives

- Represent each job as a structure holding its ID, name, deadline, profit, priority and scheduled flag.
- Sort the jobs by profit, using priority to break ties.
- Assign each job to the latest free slot on or before its deadline.
- Skip a job when no slot is available, and record that it is unscheduled.
- Report the schedule, total profit, slot utilisation and unscheduled jobs.
- Provide a simple menu so that jobs can be displayed and the schedule generated repeatedly.
- Keep the program modular, readable and easy to extend.

2.3 Scope

The project is an offline, console-based prototype. It accepts at most 20 jobs, assumes every job takes exactly one time unit, and keeps the schedule in a fixed array of 100 slots. It does not store jobs permanently, handle jobs of different lengths or run jobs in real time.

2.4 Success Criteria

The project is successful when the program accepts valid job data, produces a schedule in which no job is placed after its deadline and no slot holds two jobs, reports the correct total profit, and lists every skipped job. Tests should also confirm the tie-breaking rule and the case where every job fits.

CHAPTER 3: THEORETICAL BACKGROUND AND THE GREEDY METHOD

3.1 The Greedy Method

A greedy algorithm builds a solution step by step and, at each step, makes the choice that looks best at that moment without reconsidering it later. It works when the problem has two properties. The greedy-choice property says that a locally best choice can be part of some optimal solution. Optimal substructure says that what remains after the choice is a smaller problem of the same kind.

3.2 Job Sequencing with Deadlines

We are given jobs J1, J2, …, Jn. Job Ji has a deadline di and a profit pi, and needs one unit of time. A set S of jobs is feasible if its members can be placed in distinct time slots so that every job finishes by its deadline. The goal is to find a feasible set with the largest total profit:

$$\textit{maximise } \Sigma\, p_i \quad \textit{over all } i \in S, \ \textit{where } S \textit{ is a feasible set}$$

3.3 Why the Greedy Choice Works

Consider the most profitable job. If some optimal schedule leaves it out, either it fits in a free slot or it can replace a less profitable job already in a slot it could use; in both cases the profit does not decrease. So there is always an optimal schedule that contains the most profitable feasible job, which is the greedy-choice property.

Placing the job in the latest free slot before its deadline keeps the earlier slots open for jobs with tighter deadlines. This is why the algorithm scans from the deadline downwards instead of taking the first free slot.

3.4 Tie Breaking with Priority

Two jobs may have the same profit. Any order gives the same total profit in the classic problem, but the result would then depend on the input order. This project adds a priority from 1 to 5 and sorts equal-profit jobs by higher priority first, so the schedule is deterministic and reflects the importance of each job.

3.5 Related Concepts

Job sequencing is related to interval scheduling, activity selection and the knapsack problem. Unlike the 0/1 knapsack problem, which needs dynamic programming, the unit-time job sequencing problem has a matroid structure that lets a simple greedy rule give the optimum.

CHAPTER 4: DATA DESIGN AND JOB MODEL

4.1 The Job Structure

Every job is stored in a C++ structure named Job. Grouping the fields in one structure keeps the data about a job together, so sorting moves a whole job at once and no field can get out of step with the others.

Field	Type	Meaning
id	string	Short job identifier such as J1
name	string	Job name (one word, no spaces)
deadline	int	Last time slot in which the job may run
profit	int	Reward earned if the job is scheduled
priority	int	Importance from 1 to 5, used to break profit ties
scheduled	bool	Set to true when the job gets a slot

4.2 Storage of Jobs

The jobs are held in a fixed array, Job jobs[20]. The array gives direct access to every job by index and is simple to sort in place. A linked list would make insertion easier, but the number of jobs is small and known before scheduling starts, so an array is the better fit.

4.3 The Schedule Array

The schedule is a string array of 100 elements. Element k - 1 holds the ID of the job placed in time slot k, and an empty string means that the slot is free. After scheduling, any empty slot up to the largest deadline is printed as Idle. The size of 100 limits deadlines to 100 slots.

4.4 Input Rules

The program asks for the number of jobs and accepts values from 1 to 20; anything else prints an error and ends the program. For every job it reads the ID, name, deadline, profit and priority with the extraction operator, which is why names must not contain spaces.

4.5 Data Validation

The current version validates only the number of jobs. A more robust version should also check that every deadline lies between 1 and 100, that profits are non-negative, that priorities lie between 1 and 5 and that job IDs are unique.

CHAPTER 5: ALGORITHM I: GREEDY JOB SEQUENCING WITH DEADLINES

5.1 Principle

The algorithm treats the jobs from most to least profitable. For each job it looks for a free slot, starting at the job's own deadline and moving towards slot 1. The first free slot found is taken. If every slot from the deadline down to 1 is already occupied, the job cannot be completed in time and is skipped.

5.2 Steps

1. Sort the jobs by profit from high to low; break ties by higher priority.
2. Find the largest deadline to know how many time slots exist.
3. Mark every slot as empty and every job as unscheduled.
4. For each job in sorted order, scan slots from its deadline down to 1.
5. Assign the first empty slot, mark the job scheduled and add its profit to the total.
6. Print the schedule, the totals, the utilisation and the unscheduled jobs.

5.3 Pseudocode

```
JOB_SEQUENCING(jobs, n):
    sort jobs by profit (descending), ties by priority (descending)
    maxDeadline = largest deadline among the jobs
    schedule[1..maxDeadline] = empty
    totalProfit = 0
    scheduledCount = 0
    for each job J in sorted order:
        J.scheduled = false
        for slot = J.deadline down to 1:
            if schedule[slot] is empty:
                schedule[slot] = J.id
                J.scheduled = true
                totalProfit = totalProfit + J.profit
                scheduledCount = scheduledCount + 1
                break
    utilisation = scheduledCount / maxDeadline * 100
    return schedule, totalProfit, utilisation
```

5.4 Reports Produced

After the allocation the program prints each time slot with its job or the word Idle, then the total number of jobs, the number scheduled and unscheduled, the total profit and the utilisation. A final section lists every unscheduled job with its profit, or confirms that all jobs were scheduled.

5.5 Interpretation

The number of scheduled jobs can never exceed the largest deadline, because every job needs its own slot. Utilisation therefore tells how fully the available slots are used: 100% means no slot is idle, while a lower value shows that deadlines left some early slots unused.

CHAPTER 6: ALGORITHM II: SORTING AND SLOT ALLOCATION

6.1 Bubble Sort by Profit and Priority

The function sortJobs orders the array with bubble sort. Neighbouring jobs are swapped when the left job has less profit than the right job, or when the profits are equal and the left job has lower priority. After each pass the weakest remaining job has moved to the end.

```
SORT_JOBS(jobs, n):
    for i = 0 to n - 2:
        for j = 0 to n - i - 2:
            if jobs[j].profit < jobs[j+1].profit
               or (jobs[j].profit == jobs[j+1].profit
                   and jobs[j].priority < jobs[j+1].priority):
                swap jobs[j] and jobs[j+1]
```

Bubble sort is stable, easy to explain and fast enough for at most 20 jobs. Its cost of $O(n^2)$ comparisons is the main reason the whole program runs in quadratic time.

6.2 Slot Allocation

Allocation is the inner loop of Chapter 5. Starting at a job's deadline and moving down finds the latest free slot, which leaves the earlier slots available for jobs with tight deadlines. This is the step that makes the greedy choice safe.

6.3 Worked Example

Five jobs are used throughout the report. J1 has deadline 2 and profit 100, J2 has deadline 1 and profit 19, J3 has deadline 2 and profit 27, J4 has deadline 1 and profit 25, and J5 has deadline 3 and profit 15. After sorting by profit the order is J1, J3, J4, J2, J5, and the largest deadline is 3.

6.4 Result of the Example

The final schedule is slot 1: J3, slot 2: J1 and slot 3: J5. The total profit is 100 + 27 + 15 = 142, three of the five jobs are scheduled and the utilisation is 3 / 3 = 100%. J4 and J2 are skipped because both must finish in slot 1, which the more profitable J3 already holds.

Step	Job	Deadline	Profit	Slots tried	Result
1	J1	2	100	2	Placed in slot 2
2	J3	2	27	2, 1	Placed in slot 1
3	J4	1	25	1	Skipped, slot 1 is taken
4	J2	1	19	1	Skipped, slot 1 is taken
5	J5	3	15	3	Placed in slot 3

6.5 Why J1 Goes to Slot 2

J1 has deadline 2, so it is placed in slot 2, the latest slot it may use, even though slot 1 was also free. Keeping slot 1 open matters for jobs with a deadline of 1. For example, with job A (deadline 2, profit 10) and job B (deadline 1, profit 9), placing A in slot 2 lets B take slot 1 for a profit of 19, whereas placing A in slot 1 would leave B with no slot and a profit of only 10.

CHAPTER 7: SYSTEM DESIGN AND C++ IMPLEMENTATION

7.1 Modular Structure

The program is one source file, job_scheduler.cpp, built around the Job structure and four functions. Keeping each task in its own function makes the code easier to read and to test.

Function	Purpose
main	Reads the number of jobs and their details, then runs the menu loop
sortJobs	Bubble sort by profit, then by priority
displayJobs	Prints the jobs as a formatted table
scheduleJobs	Sorts, allocates slots and prints the schedule and statistics

7.2 Processing Flow

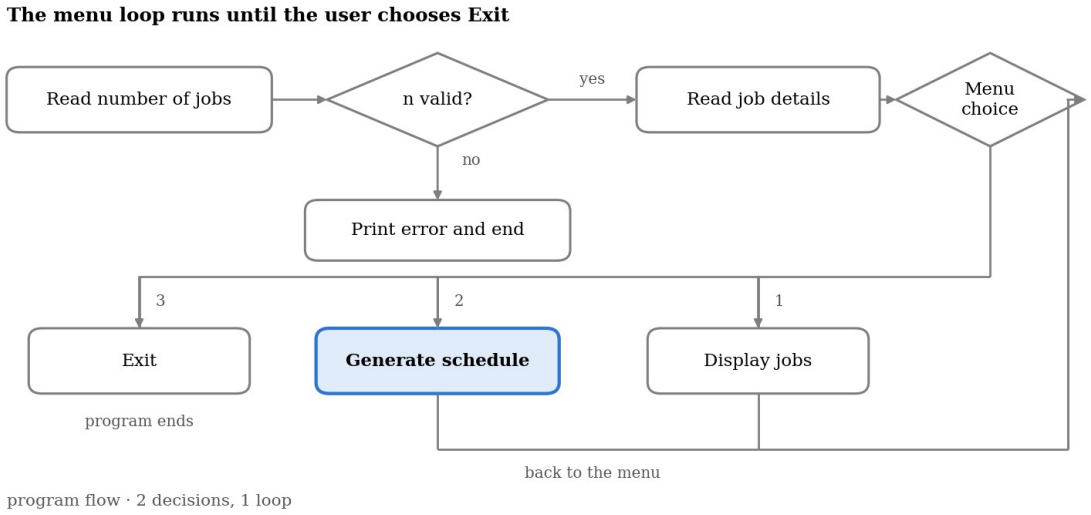


Figure 7.1 — Program flow of the Smart Job Scheduling System.

Generate Schedule is the only path that runs the greedy algorithm. Display Jobs and Generate Schedule both return to the menu, so the user can display and schedule as often as needed.

7.3 Core Allocation Code

The heart of the program is the nested loop in scheduleJobs. The outer loop visits the jobs in sorted order and the inner loop scans the slots from the deadline down to slot 1.

```
for (int i = 0; i < n; i++)
{
    for (int slot = jobs[i].deadline; slot >= 1; slot--)
    {
        if (schedule[slot - 1] == "")
        {
            schedule[slot - 1] = jobs[i].id;
            jobs[i].scheduled = true;
            totalProfit += jobs[i].profit;
            scheduledCount++;
            break;
        }
    }
}
```

The break statement ends the scan as soon as a slot is taken, so each job is placed at most once. A job whose scan finishes without finding a free slot keeps scheduled = false and is later reported as unscheduled.

7.4 Output Formatting

The iomanip header formats the output. The setw manipulator with left alignment lines up the columns of the job table, and fixed with setprecision(2) prints the utilisation with two decimal places. Utilisation is calculated as scheduledCount divided by the largest deadline, multiplied by 100.

7.5 Side Effects and Error Handling

scheduleJobs sorts the jobs array in place, so Display Jobs shows the jobs in sorted order after a schedule has been generated. An invalid number of jobs prints an error message and ends the program, and an unknown menu choice prints Invalid choice and shows the menu again.

CHAPTER 8: TESTING AND COMPARATIVE RESULTS

8.1 Purpose of Testing

Testing checks that the program returns a valid schedule, that every job is placed on or before its deadline, and that no slot holds two jobs. Six test cases were run on the submitted program, covering the worked example, jobs that all fit, jobs that compete for one slot, a profit tie, idle slots and invalid input.

8.2 Test Cases and Results

All results below are the real output of the program. Utilisation is the number of scheduled jobs divided by the largest deadline.

Test	Input summary	Slots (1, 2, 3 …)	Profit	Scheduled / total	Utilisation
T1	Worked example, 5 jobs, deadlines 2, 1, 2, 1, 3	J3, J1, J5	142	3 / 5	100.00%
T2	3 jobs with deadlines 3, 2, 1	C, B, A	60	3 / 3	100.00%
T3	4 jobs, all with deadline 1	P	50	1 / 4	100.00%
T4	2 jobs, profit 40 each, priorities 2 and 5	Y	40	1 / 2	100.00%
T5	1 job, deadline 4, profit 50	Idle, Idle, Idle, Z	50	1 / 1	25.00%
T6	Number of jobs entered as 0	No schedule	-	-	Invalid number of jobs

8.3 Observations

In T1 the two skipped jobs are J4 and J2, both with deadline 1. In T3 only the most profitable job, P with profit 50, survives, because all four jobs need slot 1. In T4 the two jobs earn the same profit, and job Y with priority 5 wins the slot over job X with priority 2, which confirms the tie-breaking rule. T5 shows that slots before a lone late job stay idle, so utilisation falls to 25%.

8.4 Comparison with Input-Order Scheduling

To see how much sorting matters, the five jobs of T1 were also allocated in their input order (J1, J2, J3, J4, J5) with the same latest-free-slot rule. J1 takes slot 2 and J2 takes slot 1, so J3 and J4 find no slot, and J5 takes slot 3.

Method	Jobs scheduled	Total profit
Input order	J2, J1, J5	134
Sorted by profit (this project)	J3, J1, J5	142

Sorting by profit gains 8 units, about 5.6% of the greedy total. The greedy result of 142 is also the best possible here: the three slots can hold at most J1 (100), then the best job for slot 1 (J3 with 27) and J5 (15) for slot 3.

8.5 Evidence to Capture

Run the program in a terminal, enter the jobs of T1, choose option 1 and then option 2, and take a screenshot showing the job table, the schedule and the totals. Use only the real output of your own run.

CHAPTER 9: SAMPLE EXECUTION AND EVIDENCE

9.1 Compiling and Running

The program uses only standard headers, so it compiles with any C++ compiler. With g++ the commands are:

```
g++ job_scheduler.cpp -o job_scheduler
./job_scheduler
```

The source compiled without errors or warnings when built with g++ using the -Wall option.

9.2 Input Used

The run below uses the five jobs of the worked example. Priorities were chosen between 1 and 5.

ID	Name	Deadline	Profit	Priority
J1	Backup	2	100	3
J2	Report	1	19	2
J3	Email	2	27	4
J4	Update	1	25	5
J5	Cleanup	3	15	1

9.3 Output of Display Jobs (Menu Option 1)

```
Available Jobs
ID   Name      Deadline   Profit   Priority
-------------------------------------------------
J1   Backup    2          100      3
J2   Report    1          19       2
J3   Email     2          27       4
J4   Update    1          25       5
J5   Cleanup   3          15       1
```

9.4 Output of Generate Schedule (Menu Option 2)

```
Smart Job Schedule
Time Slot 1 : J3
Time Slot 2 : J1
Time Slot 3 : J5

Total Jobs       : 5
Scheduled Jobs   : 3
Unscheduled Jobs : 2
Total Profit     : 142
Utilization      : 100.00%

Unscheduled Jobs
J4 - Update | Profit: 25
J2 - Report | Profit: 19
```

The schedule matches the hand calculation in Chapter 6: J3, J1 and J5 fill the three slots for a profit of 142, and J4 and J2 are reported as unscheduled in sorted order.

9.5 Output with Idle Slots (Test T5)

```
Smart Job Schedule
Time Slot 1 : Idle
Time Slot 2 : Idle
Time Slot 3 : Idle
Time Slot 4 : Z

Total Jobs       : 1
Scheduled Jobs   : 1
Unscheduled Jobs : 0
Total Profit     : 50
Utilization      : 25.00%

Unscheduled Jobs
All jobs were scheduled successfully.
```

9.6 Screenshot

Insert here a screenshot of your own terminal run of the program, showing the command, the job table and the schedule with its totals. The text outputs above were captured from the program; the typed answers do not appear in them because the input was supplied from a file.

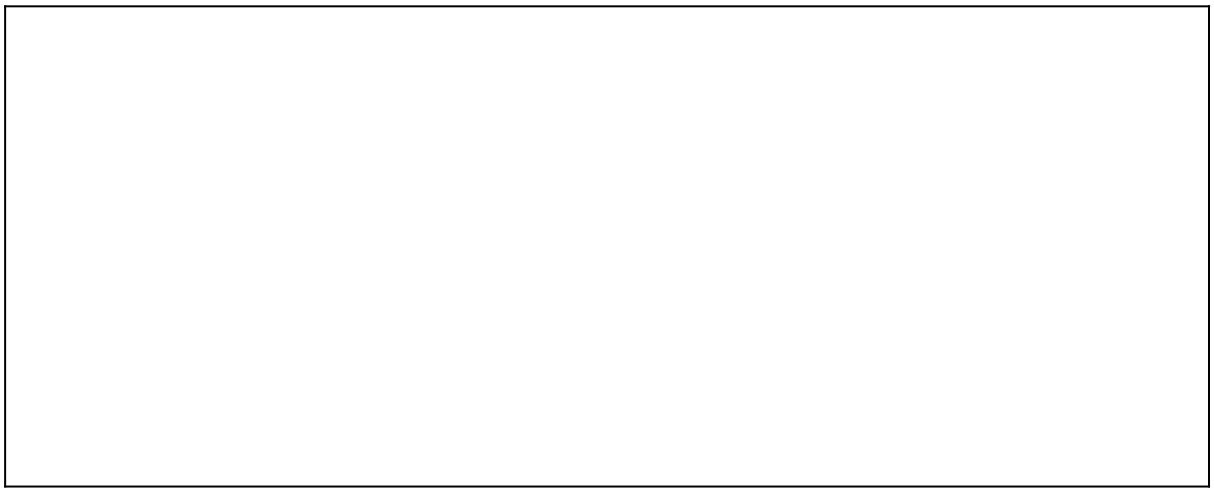


Figure 9.1 — Terminal run of the Smart Job Scheduling System.

9.7 Reproducibility

A reproducible demonstration records the compile command, the jobs entered, the menu options chosen and the resulting schedule. Running the same input always gives the same schedule, because the sort and the slot search contain no randomness.

CHAPTER 10: COMPLEXITY ANALYSIS AND CORRECTNESS

10.1 Time Complexity

Let n be the number of jobs and d the largest deadline. The table shows the cost of each stage of scheduleJobs.

Stage	Time
Bubble sort by profit and priority	$O(n^2)$
Finding the largest deadline and resetting flags	$O(n)$
Clearing the schedule array of 100 slots	$O(1)$, a fixed size
Slot allocation for all jobs	$O(n \cdot d)$
Printing slots, totals and unscheduled jobs	$O(d + n)$
Whole algorithm	$O(n^2 + n \cdot d)$

Because a job's scan can cover at most d slots, allocation costs at most $n \cdot d$ steps. When d is about n, the whole algorithm runs in $O(n^2)$.

10.2 Space Complexity

The schedule array needs O(d) space and the jobs array needs O(n). The program also uses a few integer variables, so the extra space beyond the input is O(d). In this program n is at most 20 and the schedule array is fixed at 100 elements.

10.3 Feasibility

The output is always a valid schedule. A job is placed only in a slot between 1 and its own deadline, so it never finishes late. A slot is used only when it is empty, so no slot holds two jobs.

10.4 Optimality

The greedy order is by decreasing profit. Suppose an optimal schedule differs from the greedy one at the first job where they disagree. If the greedy job is missing from the optimal schedule, it can replace a job of equal or lower profit that occupies a slot it could use, or take a free slot, without lowering the total. Repeating this exchange turns the optimal schedule into the greedy schedule with no loss of profit, so the greedy schedule is optimal.

Choosing the latest free slot is what keeps this argument valid: it leaves the early slots free for jobs with tighter deadlines, as shown in Section 6.5.

10.5 Faster Variants

The quadratic cost comes from bubble sort and from scanning slots one at a time. Sorting with merge sort takes O(n log n). The scan for the latest free slot can be replaced with a Union-Find structure that points every taken slot to the next free slot below it, which makes each lookup almost constant time. Together these changes give a running time of O(n log n).

10.6 Practical Performance

With at most 20 jobs and 100 slots, the program finishes instantly. The comparison of methods is therefore about correctness and clarity, not speed. To study scaling, the algorithm could be tested on generated job sets of growing size while recording the running time.

CHAPTER 11: LIMITATIONS, FUTURE SCOPE AND CONCLUSION

11.1 Limitations

The program handles at most 20 jobs and keeps the schedule in an array of 100 slots. A deadline above 100 would make the allocation loop read and write outside that array, so deadlines must be kept within 1 to 100. Only the number of jobs is validated; negative profits, priorities outside 1 to 5, duplicate job IDs and non-numeric input are not checked.

Job names are read with the extraction operator, so they cannot contain spaces. Every job needs exactly one time unit, which does not match real tasks of different lengths. Jobs exist only while the program runs, and scheduleJobs reorders the job array in place.

11.2 Future Enhancements

- Replace the fixed arrays with std::vector so the number of jobs and slots is unlimited.
- Add input validation for deadlines, profits, priorities and duplicate IDs.
- Use merge sort and Union-Find to reach O(n log n) running time.
- Read names with getline so that they may contain spaces.
- Save jobs to a file and load them again.
- Support jobs with different durations, which turns the problem into a weighted interval scheduling task solved with dynamic programming.
- Add several machines and a graphical interface that draws the schedule as a timeline.

11.3 Conclusion

The Smart Job Scheduling System shows how a greedy strategy solves job sequencing with deadlines. Sorting the jobs by profit, breaking ties by priority and placing each job in the latest free slot before its deadline gives the maximum total profit for unit-time jobs. In the worked example the schedule J3, J1, J5 earns 142, eight units more than scheduling the jobs in input order.

The main learning outcome is that a greedy algorithm is correct only when its choice rule is justified. Here, taking the most profitable job first and giving it the latest slot is safe, and the same idea fails if the earliest slot is used. Clear modelling, a modular implementation, complexity analysis and tests on the real program make the result reliable.

REFERENCES AND APPENDIX

References

1. Mayuri Dudhat. Smart Job Scheduling System, C++ source file job_scheduler.cpp, DAA project, 2026.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). Introduction to Algorithms, 3rd edition. MIT Press. See the chapter on greedy algorithms and the section on task scheduling.
3. Horowitz, E., Sahni, S., and Rajasekaran, S. Fundamentals of Computer Algorithms, 2nd edition. Universities Press. See the greedy method chapter, job sequencing with deadlines.
4. Levitin, A. (2011). Introduction to the Design and Analysis of Algorithms, 3rd edition. Pearson.
5. C++ reference documentation for iostream, iomanip and string. https://en.cppreference.com

Appendix A — Core Source Code (job_scheduler.cpp)

```
struct Job
{
    string id;
    string name;
    int deadline;
    int profit;
    int priority;
    bool scheduled;
};

void sortJobs(Job jobs[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (jobs[j].profit < jobs[j + 1].profit ||
                (jobs[j].profit == jobs[j + 1].profit &&
                 jobs[j].priority < jobs[j + 1].priority))
            {
                Job temp = jobs[j];
                jobs[j] = jobs[j + 1];
                jobs[j + 1] = temp;
            }
        }
    }
}

void scheduleJobs(Job jobs[], int n)
{
    sortJobs(jobs, n);

    int maxDeadline = 0;
    for (int i = 0; i < n; i++)
    {
        jobs[i].scheduled = false;
```

```
        if (jobs[i].deadline > maxDeadline)
            maxDeadline = jobs[i].deadline;
    }

    string schedule[100];
    for (int i = 0; i < 100; i++)
        schedule[i] = "";

    int totalProfit = 0;
    int scheduledCount = 0;

    for (int i = 0; i < n; i++)
    {
        for (int slot = jobs[i].deadline; slot >= 1; slot--)
        {
            if (schedule[slot - 1] == "")
            {
                schedule[slot - 1] = jobs[i].id;
                jobs[i].scheduled = true;
                totalProfit += jobs[i].profit;
                scheduledCount++;
                break;
            }
        }
    }
    // ... schedule, totals and unscheduled jobs are then printed
}
```

The functions displayJobs and main are not repeated here. The complete program is submitted with this report as job_scheduler.cpp.

Appendix B — Final Submission Checklist

- Add the real terminal screenshot in Section 9.6.
- Check the faculty guide, lab teacher and head of department names on the cover and certificate.
- Fill in the date on the certificate and declaration.
- Run the program once more and confirm the outputs in Chapters 8 and 9.
- Update page numbers and the table of contents after all edits.